

oom-watch — Architecture and Design Decisions

Why it is the way it is

OOM-Protect maintainers

2026-04-30

oom-watch — Architecture

- Single-sentence summary
- Component diagram
- Why these decisions
 - Why Go and not bash
 - Why atop and not /proc directly
 - Why stdlib only (no yaml.v3, no testify)
 - Why JSON config and not YAML
 - Why an `OnIncident` callback instead of monitor importing snapshot
 - Why filter PRM at the display layer instead of the parser
 - Why forensic enrichment runs at snapshot time, not at report-read time
 - Why install validates with `-dry-run` BEFORE enabling the unit
 - Why “best-effort” snapshot capture
 - Why atomic writes
 - Why the cooldown bypasses on escalation
- Threshold rationale
- Anti-bluff testing — applied
- What this daemon does *not* do
- Future work
- See also

oom-watch — Architecture

This document explains the *why* behind the daemon. The user manual (`manuals/oom-watch-manual.md`) explains the *how*.

Single-sentence summary

`oom-watch` is an idempotent, stdlib-only Go daemon that periodically asks atop for one parseable sample, runs it through a pure threshold engine, and — if the verdict is non-zero and the cooldown has elapsed (or an escalation has occurred) — atomically writes a Markdown forensic report assembled from the sample plus best-effort `/proc`, `cgroup`, and `journal` captures.

Component diagram



Why these decisions

Why Go and not bash

`oom-hardening.sh` and `oom-runner.sh` are bash because they are *one-shot installers / wrappers* — they need to be reviewable as plain text by a sysadmin. `oom-watch` is *long-running and stateful* (cooldowns, severity tracking) and must run without operator attention for weeks. Bash doesn't give you typed structs, a stdlib HTTP/JSON parser that won't panic, or `slog`. Go does, and produces a single statically-built binary.

Why atop and not /proc directly

Three reasons. First, atop already does the hard work of consistent sampling, command-name handling (the `(cmd with spaces)` form), and per-process accounting (PRM, PRC). Re-implementing that would be a multi-month project with many subtle bugs. Second, atop is already mandatory in this environment because the post-mortem (`reports/Crash_Report.md`) recommends it. Third, atop's parseable output (`-PALL`) is stable and documented in the man page; we depend on a contract atop maintains, not on internals.

Why stdlib only (no yaml.v3, no testify)

Reproducibility. The binary must build offline, on any machine that has Go 1.22+, with no `go mod tidy` network call. JSON is universal; sysadmins are comfortable editing it. testify is convenient but every assertion it provides can be written in 5 lines of stdlib `t.Errorf`.

Why JSON config and not YAML

JSON has zero ambiguity (no Norway problem, no significant whitespace), is in the stdlib, and the full config is < 30 lines. YAML's only advantage — comments — is replaced here by aliased keys (`_comment`) plus the example file being heavily commented.

Why an `OnIncident` callback instead of monitor importing snapshot

Acyclic dependencies. `snapshot.Snapshot` has a `Verdict` field of type `monitor.Verdict`. If `monitor` then imported `snapshot` to call `Capture`, we'd have a cycle. The cleanest fix is to make `monitor` declare the *callback shape* and have `cmd/oomwatch/main.go` wire `snapshot.Capture` and `report.WriteMarkdown` together.

Why filter PRM at the display layer instead of the parser

atop 2.x emits one PRM row per kernel thread; non-leader rows duplicate the parent's RSize (because threads share memory) and would otherwise drown out every real process in the top-N list. The fix lives in `snapshot.topProcesses`, not in the parser, for two reasons:

1. **Forensic completeness.** A future report (e.g. per-thread CPU breakdown) may want every PRM row. Discarding non-leaders during parse is information loss; discarding them during display is reversible.

2. **Version safety.** atop 1.x does not emit the leader flag. The parser sets `IsLeader=true` as a default, so ancient atop hosts still produce useful top-mem tables instead of empty ones — a parser-time filter would silently drop everything.

Tests: `TestParse_RealFixture` asserts `IsLeader` is decoded correctly for `y`, `n`, and old-style (no flag) rows. `TestTopProcesses_FiltersNonLeaders` asserts the snapshot drops non-leaders before they reach the report. `challenge-real-atop.sh` asserts no PID appears more than once in the top-mem table when running real atop on the host.

Why forensic enrichment runs at snapshot time, not at report-read time

atop's PRM label gives a 15-character truncated command name and no context — for root-cause work an operator needs the full `/proc/<pid>/cmdline`, parent PID, owning UID, cgroup path, peak RSS, and `oom_score`. There are two places this enrichment could live:

1. **At report-read time** — the daemon writes only the basics; an operator runs a separate tool to enrich a saved report.
2. **At snapshot time** — the daemon reads `/proc/<pid>/*` for the top-N PIDs as part of `Capture()` and stores the enriched detail in the report.

We chose (2). Two reasons:

- **The PID may not exist later.** `/proc/<pid>/cmdline` only exists while the process is running. By the time an operator reads the report 30 minutes later, the runaway may already have been OOM-killed or restarted with a new PID, and the cmdline is gone forever. The whole point of a forensic report is preserving evidence at the moment it matters.
- **No external tool to maintain.** A read-time enricher would be a separate binary that drifts out of sync with the report format.

The cost is bounded: enrichment caps at `EnrichTopN=10` PIDs by default, each requiring 4–5 small `/proc` reads (a few hundred microseconds total). On a host with 2000+ processes, that's negligible compared to the atop sampling itself.

Why install validates with `-dry-run` BEFORE enabling the unit

A subtle bluff hit us once: the shipped `oom-watch.example.json` carried a `_comment` JSON field for self-documentation, but `config.Load` uses `DisallowUnknownFields()` to reject typos. The example was thus a config the parser refused. `make oomwatch-install` copied it verbatim to `/etc/oom-watch/config.json`, and the systemd unit cycled `activating → failed` with `exit-code=2` — accurate, but useless to the operator who didn't know which exit code maps to which failure class.

`install-and-verify.sh` now runs `oomwatch -dry-run -config /etc/oom-watch/config.json` between *install* and *enable*. A failure surfaces the validator's exact diagnostic ("unknown field `_comment`" / "memory_used_ratio: warn must be ≤ critical" / etc.) plus a remediation paragraph, and the script aborts before touching systemd.

`challenges/challenge-config-validation.sh` carries the regression: a final assertion runs `oom-watch -dry-run` against the shipped `oom-watch.example.json`. A future commit that re-introduces an unknown field there will fail the test, not the user's first install.

Why “best-effort” snapshot capture

A perfect-or-nothing snapshot would miss the most important reports — the ones produced under degraded conditions where `/proc/pressure` is unreadable or `journalctl` times out. A report with 8 of 10 sections + a `Capture errors` section that names the missing two is far more useful than no report at all. Tests enforce this: `TestCapture_PartialFailure` asserts the snapshot is non-nil and `Errors` lists the missing files.

Why atomic writes

If the daemon is killed mid-write (the OOM scenario itself!) we must not leave a half-written `.md` that downstream tooling will misparse. We write to `.oom-watch-*.tmp` in the same directory and `os.Rename` on success. `TestWriteMarkdown_NoLeftoverTemp` verifies this.

Why the cooldown bypasses on escalation

Without it, the most important transition — `WARN` → `CRITICAL` — would be silently dropped if it occurred within the warn-level cooldown window. That’s exactly when you most need a report. `TestLoop_EscalationBypassesCooldown` verifies this.

Threshold rationale

Metric	Default	Why
<code>memory_used_ratio_critical</code>	0.95	Below this the kernel can usually reclaim from caches without thrashing. At 0.95 the next allocation is likely to swap or OOM.
<code>psi_mem_full_avg10_critical</code>	30.0	“All non-idle tasks fully stalled on memory $\geq 30\%$ of the last 10 seconds” — the system is no longer making forward progress.
<code>psi_mem_full_avg10_warn</code>	10.0	Detectable user-visible slowness begins around here.
<code>swap_used_ratio_critical</code>	0.80	Beyond 80% swap, swapping latency dominates and any further pressure means OOM kills.
<code>load_per_cpu_critical</code>	4.0	Sustained 4× saturation on per-CPU load is not transient queueing; something is genuinely overloaded.

These mirror the post-mortem in `reports/Crash_Report.md` and the systemd-oomd defaults set by `oom-hardening.sh`.

Anti-bluff testing — applied

Every package has at least one test that asserts a specific value derived from a real fixture. Examples:

- `TestParse_RealFixture` asserts `MEM.AvailPages == 4500000` and `PSI.MemFullAvg10 == 45.20` — both extracted from the fixture by hand. A parser that returned zero values would fail.
- `TestCapture_FakeProc` writes specific bytes to a fake `/proc/meminfo` and asserts those bytes appear in the snapshot.

- `TestWriteMarkdown_FullReport` writes a known Snapshot, reads the file back, and asserts every required section heading appears in the body.

The mutation Challenge (`challenge-tests-are-not-bluffs.sh`) breaks `AvailPages: pi(24)` to `AvailPages: 0` , runs the suite, asserts non-zero exit, restores. If a future contributor rewrites the parser as a no-op, the Challenge will fail.

What this daemon does *not* do

- **It does not kill processes.** Killing is the job of `systemd-oomd` (configured by `oom-hardening.sh`) and the kernel OOM-killer. oom-watch only writes reports.
- **It does not page or alert externally.** No email, no Slack, no PagerDuty. Reports are local artifacts; integrate with your alerting tool of choice (an `inotifywait` watcher on `/var/log/oom-watch/reports/` is a typical first step).
- **It does not rotate or compress reports.** Use `find ... -mtime +30 -delete` or logrotate.
- **It does not run as non-root.** It needs to read `/proc//stat` for every process and the cgroup tree; doing that as a non-root user requires per-process permissions that are not portable.

Future work

- **netatop-bpf integration** — atop's optional `netatop` module emits PRN (per-process network) lines. Wire those into the snapshot when present.
- **Memory.events delta tracking** — read `/sys/fs/cgroup/.../memory.events` and report `oom_kill` / `oom` deltas since previous sample.
- **HTTP endpoint** — a `--listen :9999` mode that serves the latest 100 reports as a JSON index. The `atophttpd` project shows the pattern.
- **Plot generation** — convert atop logs around the incident into PNG plots, link from the report. The `atopsar-plot` project is a starting reference.

See also

- `Constitution.md` — anti-bluff testing charter (Article I).
- `manuals/oom-watch-manual.md` — user-facing operations guide.
- `reports/Crash_Report.md` — the 2026-04-28 incident this toolkit answers.
- `oom-watch/README.md` — submodule entry point.